

BUKU AJAR

KRIPTOGRAFI

Untuk Program Studi Matematika S1

Mata kuliah Kriptografi | Kode MAT-XXX | 3 SKS

Program Studi Matematika | Fakultas MIPA

ISBN: *(diisi sesuai kebijakan institusi)*

Berbasis Outcome-Based Education (OBE)

Departemen Matematika

30 Januari 2026

Buku Ajar Kriptografi

Untuk Program Studi Matematika S1

Berbasis Rencana Pembelajaran Semester (RPS)

14 Bab | 3 SKS

Mata kuliah Kriptografi | Kode MAT-XXX | 3 SKS

Program Studi Matematika | Fakultas MIPA

Penerbit: *Nama Institusi, Kota, Tahun*

Departemen Matematika

Hak Cipta

Hak cipta dilindungi undang-undang. Dilarang mengutip atau memperbanyak sebagian maupun seluruh isi buku ini tanpa izin tertulis dari penerbit.

Penulis

Nama Penulis

Departemen Matematika, *Nama Fakultas/Universitas*

Penelaah

Nama Penelaah

Afiliasi

Editor

Nama Editor (atau — jika tidak ada)

Penerbit

Nama Institusi/Unit, Kota, Tahun

Tahun Terbit / Copyright

Tahun

Kata Pengantar

Puji syukur ke hadirat Tuhan Yang Maha Esa atas terselesaikannya Buku Ajar Kriptografi ini. Buku ini disusun sebagai bahan ajar mata kuliah Kriptografi untuk Program Studi Matematika jenjang Strata 1 (S1), berdasarkan Silabus dan Rencana Pembelajaran Semester (RPS) mata kuliah Kriptografi serta Capaian Pembelajaran Mata Kuliah (CPMK) Program Studi Matematika yang berlaku.

Materi dalam buku ini disusun mengikuti alur pembelajaran semester dengan 14 bab yang sesuai dengan 14 pertemuan efektif (tidak termasuk Ujian Tengah Semester dan Ujian Akhir Semester). Setiap bab mencakup tujuan pembelajaran, penyajian materi, dan rangkuman. Implementasi algoritma kriptografi disajikan dalam empat bahasa pemrograman: MATLAB, C, C++, dan Pascal, sesuai dengan tuntutan kurikulum.

Penulis berharap buku ajar ini dapat membantu mahasiswa dalam mempelajari kriptografi secara sistematis, baik dari aspek matematika maupun implementasi praktis.

Saran dan masukan untuk perbaikan edisi selanjutnya sangat diharapkan.

Penulis

Daftar Isi

Kata Pengantar	i
Daftar Gambar	vii
Daftar Tabel	ix
1 Pengantar Kriptografi	1
1.1 Pendahuluan	1
1.2 Model Komunikasi Aman	2
1.3 Ancaman Keamanan	3
1.4 Confidentiality, Integrity, dan Authenticity	3
1.5 Kriptografi Simetris dan Asimetris	4
1.5.1 Kriptografi Simetris	5
1.5.2 Kriptografi Asimetris	5
1.5.3 Kriptografi Hibrida	5
2 Aritmetika Modular	7
2.1 Aritmetika Modular	7
2.1.1 Invers Modular	8
3 Algoritma Euclid dan Euclid Diperluas	9
3.1 Algoritma Euclid dan Euclid Diperluas	9

3.1.1	Algoritma Euclid	9
3.1.2	Algoritma Euclid Diperluas	9
4	Teori Bilangan: Prima dan Fast Exponentiation	11
4.1	Teori Bilangan	11
4.1.1	Bilangan Prima	11
4.1.2	Teorema Fermat Kecil	12
4.1.3	Fungsi Euler	12
4.2	Pangkat Modular (Fast Exponentiation)	12
5	Cipher Caesar dan Substitusi Sederhana	15
5.1	Cipher Caesar	15
5.2	Substitusi Monoalfabetik	17
6	Cipher Vigenère dan Polyalphabetic	19
6.1	Cipher Vigenère	19
6.2	Transposisi	20
7	Block Cipher: DES, AES, dan Mode Operasi	21
7.1	Block Cipher, DES, dan AES	21
7.1.1	Block Cipher	21
7.1.2	DES	22
7.1.3	AES	22
7.2	Mode Operasi Block Cipher	22
7.2.1	ECB (Electronic Codebook)	23
7.2.2	CBC (Cipher Block Chaining)	23
8	Algoritma RSA	25
8.1	Konsep Kunci Publik	25

8.2	Algoritma RSA	26
8.2.1	Generasi Kunci	26
8.2.2	Enkripsi dan Dekripsi	26
9	Fungsi Hash (SHA, MD5)	29
9.1	Fungsi Hash	29
9.2	SHA dan MD5	30
9.2.1	MD5	30
9.2.2	SHA (Secure Hash Algorithm)	30
10	Tanda Tangan Digital	31
10.1	Tanda Tangan Digital	31
11	Protokol Kriptografi	33
11.1	Protokol Kriptografi	33
11.2	Protokol Diffie-Hellman	33
12	Manajemen Kunci	35
12.1	Manajemen Kunci	35
13	Kriptanalisis	37
13.1	Frequency Analysis	37
13.2	Kasiski Examination	38
13.3	Serangan Kriptanalisis	38
14	Implementasi Multi-Bahasa	41
14.1	Implementasi dalam MATLAB	41
14.1.1	Caesar Cipher	41
14.1.2	Algoritma Euclid Diperluas	42

14.2 Implementasi dalam C	43
14.2.1 Caesar Cipher	43
14.2.2 Fast Modular Exponentiation	44
14.3 Implementasi dalam C++	45
14.4 Implementasi dalam Pascal	46

Daftar Referensi	49
-------------------------	-----------

Daftar Gambar

1.1	Model komunikasi kriptografi: Alice mengirim pesan terenkripsi ke Bob; Eve menyadap saluran.	2
1.2	Kriptografi simetris: satu kunci untuk enkripsi dan dekripsi	5
8.1	Alur enkripsi RSA: Alice mengenkripsi dengan kunci publik Bob	26
10.1	Alur tanda tangan digital berbasis RSA	31
11.1	Protokol Diffie-Hellman: pertukaran nilai untuk menghasilkan kunci rahasia	34

Daftar Tabel

1.1	Jenis serangan kriptografi berdasarkan informasi yang dimiliki penyerang .	3
1.2	Triad CIA dan mekanisme kriptografis	4
1.3	Perbandingan kriptografi simetris dan asimetris	5
2.1	Operasi aritmetika modular	7
4.1	Contoh: $3^{11} \bmod 5 = 2$ ($11 = 1011_2$)	12
5.1	Tabel Caesar cipher dengan shift 3 (A→D, B→E, ...)	16
6.1	Contoh Vigenère: plaintext “HELLO”, kunci “KEY”	19
7.1	Perbandingan DES dan AES	22
9.1	Sifat fungsi hash kriptografis	29
13.1	Frekuensi huruf dalam bahasa Inggris (%)	37

Bab 1

Pengantar Kriptografi

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menjelaskan model komunikasi aman dan ancaman keamanan
- Mendefinisikan confidentiality, integrity, dan authenticity
- Membedakan kriptografi simetris dan asimetris

1.1 Pendahuluan

Kriptografi merupakan ilmu dan seni untuk menjaga kerahasiaan pesan dengan cara menyandikan pesan tersebut sehingga hanya pihak yang berwenang yang dapat membacanya [1]. Secara etimologi, kata “kriptografi” berasal dari bahasa Yunani: *kryptos* (tersembunyi) dan *graphein* (menulis). Dengan demikian, kriptografi dapat diartikan sebagai “tulisan tersembunyi”.

Dalam era digital saat ini, kriptografi menjadi fondasi utama keamanan informasi [2]. Setiap aktivitas komunikasi daring—mulai dari pesan instan, transaksi perbankan, hingga penyimpanan data—mengandalkan teknik kriptografi untuk melindungi data dari pihak yang tidak berwenang.

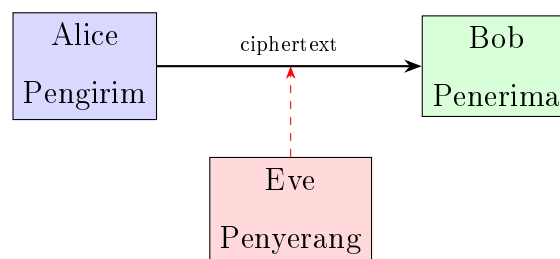
Buku ajar ini dirancang untuk mahasiswa Program Studi Matematika jenjang S1 yang

mempelajari kriptografi sebagai mata kuliah wajib atau pilihan. Materi disusun secara sistematis mulai dari konsep dasar, matematika pendukung, algoritma klasik dan modern, hingga implementasi dalam berbagai bahasa pemrograman (MATLAB, C, C++, dan Pascal).

1.2 Model Komunikasi Aman

Model dasar komunikasi aman melibatkan beberapa pihak utama:

- **Pengirim (Sender/Alice):** Pihak yang mengirim pesan.
- **Penerima (Receiver/Bob):** Pihak yang menerima pesan.
- **Penyerang (Attacker/Eve):** Pihak yang mencoba menyadap atau memodifikasi pesan tanpa izin.
- **Sumber terpercaya (Trusted Authority):** Pihak yang mendistribusikan kunci atau sertifikat.



Gambar 1.1: Model komunikasi kriptografi: Alice mengirim pesan terenkripsi ke Bob; Eve menyadap saluran.

Dalam model klasik Shannon [3], Alice ingin mengirim pesan rahasia ke Bob melalui saluran yang tidak aman. Alice mengenkripsi pesan (plaintext) menjadi ciphertext menggunakan kunci, kemudian Bob mendekripsi ciphertext menjadi plaintext menggunakan kunci yang sama atau terkait. Eve yang menyadap saluran hanya melihat ciphertext dan tidak dapat memperoleh informasi bermakna tanpa kunci [4].

1.3 Ancaman Keamanan

Ancaman keamanan terhadap sistem kriptografi dapat diklasifikasikan berdasarkan jenis serangan:

Tabel 1.1: Jenis serangan kriptografi berdasarkan informasi yang dimiliki penyerang

Jenis Serangan	Informasi Penyerang	Tingkat Kesulitan
Ciphertext-only	Hanya ciphertext	Paling sulit bagi penyerang
Known-plaintext	Pasangan plaintext–ciphertext	Sedang
Chosen-plaintext	Dapat memilih plaintext, dapat ciphertext	Lebih mudah
Chosen-ciphertext	Dapat memilih ciphertext, dapat plaintext	Paling mudah bagi penyerang

1. **Serangan ciphertext-only:** Penyerang hanya memiliki akses ke ciphertext. Ini adalah skenario terlemah bagi penyerang.
2. **Serangan known-plaintext:** Penyerang memiliki pasangan plaintext–ciphertext dan mencoba memperoleh kunci.
3. **Serangan chosen-plaintext:** Penyerang dapat memilih plaintext dan memperoleh ciphertext yang sesuai.
4. **Serangan chosen-ciphertext:** Penyerang dapat memilih ciphertext dan memperoleh plaintext yang sesuai.

Selain itu, ancaman juga mencakup **man-in-the-middle** (penyerang menyisipkan diri antara Alice dan Bob), **replay attack** (mengulang pesan yang pernah dikirim), dan **denial of service** (membuat layanan tidak tersedia) [5, 6].

1.4 Confidentiality, Integrity, dan Authenticity

Tiga tujuan utama keamanan informasi dikenal sebagai triad CIA:

Tabel 1.2: Triad CIA dan mekanisme kriptografis

Tujuan	Deskripsi	Mekanisme
Confidentiality	Hanya pihak berwenang yang dapat mengakses	Enkripsi simetris/asimetris
Integrity	Data tidak diubah tanpa sepengetahuan	Fungsi hash, MAC, tanda tangan
Authenticity	Memastikan identitas pengirim	Tanda tangan digital, sertifikat
Non-repudiation	Pengirim tidak dapat menyangkal	Tanda tangan digital

1. **Confidentiality (Kerahasiaan):** Memastikan informasi hanya dapat diakses oleh pihak yang berwenang. Enkripsi adalah mekanisme utama untuk mencapai kerahasiaan.
2. **Integrity (Integritas):** Memastikan informasi tidak diubah oleh pihak yang tidak berwenang. Fungsi hash dan tanda tangan digital digunakan untuk memverifikasi integritas.
3. **Authenticity (Keaslian):** Memastikan identitas pengirim dan bahwa pesan benar-benar berasal dari pihak yang mengklaim. Autentikasi dan tanda tangan digital menyediakan keaslian.

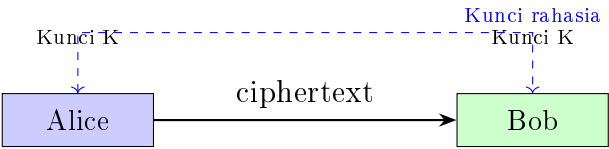
Sering kali ditambahkan **Non-repudiation (Nirpenyangkalan):** Memastikan pengirim tidak dapat menyangkal telah mengirim pesan. Tanda tangan digital memberikan bukti yang tidak dapat disangkal [2, 7].

1.5 Kriptografi Simetris dan Asimetris

Berdasarkan penggunaan kunci, kriptografi dibagi menjadi dua kategori utama:

Tabel 1.3: Perbandingan kriptografi simetris dan asimetris

Aspek	Simetris	Asimetris
Jumlah kunci	1 kunci rahasia	2 kunci (publik + privat)
Enkripsi/Dekripsi	Kunci sama	Kunci berbeda
Contoh	AES, DES, Caesar	RSA, ElGamal
Kecepatan	Cepat	Lebih lambat
Distribusi kunci	Sulit (perlu saluran aman)	Mudah (kunci publik terbuka)



Gambar 1.2: Kriptografi simetris: satu kunci untuk enkripsi dan dekripsi

1.5.1 Kriptografi Simetris

Menggunakan **satu kunci** yang sama untuk enkripsi dan dekripsi [1, 8]. Alice dan Bob harus berbagi kunci rahasia secara aman sebelum berkomunikasi. Contoh: Caesar cipher, AES, DES. Kelebihan: cepat dan efisien. Kelemahan: masalah distribusi kunci—bagaimana mengirim kunci secara aman?

1.5.2 Kriptografi Asimetris

Menggunakan **sepasang kunci**: kunci publik (untuk enkripsi) dan kunci privat (untuk dekripsi). Kunci publik dapat didistribusikan secara terbuka; hanya kunci privat yang dijaga rahasia. Contoh: RSA. Kelebihan: tidak perlu saluran aman untuk pertukaran kunci. Kelemahan: lebih lambat dibanding kriptografi simetris [6].

1.5.3 Kriptografi Hibrida

Dalam praktik, sistem modern sering menggabungkan keduanya: kriptografi asimetris untuk pertukaran kunci, lalu kriptografi simetris untuk mengenkripsi data aktual (misalnya TLS/SSL).

Rangkuman

Bab ini memperkenalkan konsep dasar kriptografi, model komunikasi aman, ancaman keamanan, triad CIA (confidentiality, integrity, authenticity), serta perbedaan kriptografi simetris dan asimetris. Pemahaman terhadap materi ini menjadi fondasi untuk mempelajari algoritma kriptografi pada bab-bab selanjutnya.

Bab 2

Aritmetika Modular

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menghitung operasi aritmetika modular (penjumlahan, perkalian, invers)
- Menerapkan kongruensi modulo dalam konteks kriptografi

2.1 Aritmetika Modular

Aritmetika modular adalah dasar matematika untuk banyak algoritma kriptografi [8, 5]. Dua bilangan bulat a dan b dikatakan **kongruen modulo n** , ditulis $a \equiv b \pmod{n}$, jika n membagi $(a - b)$.

Tabel 2.1: Operasi aritmetika modular

Operasi	Rumus
Penjumlahan	$(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
Pengurangan	$(a - b) \bmod n = ((a \bmod n) - (b \bmod n) + n) \bmod n$
Perkalian	$(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n$

2.1.1 Invers Modular

Bilangan a^{-1} adalah invers modular dari a modulo n jika $a \cdot a^{-1} \equiv 1 \pmod{n}$. Invers modular hanya ada jika $\gcd(a, n) = 1$. Algoritma Euclid diperluas digunakan untuk menghitung invers modular [8, 5].

Listing 2.1: Operasi modular dalam MATLAB

```
1 % Contoh: 17 mod 5 = 2
2 mod(17, 5)    % hasil: 2
3
4 % Perkalian modular: (12 * 7) mod 10
5 mod(12 * 7, 10) % hasil: 4
6
7 % Pangkat modular: 2^10 mod 1000
8 mod(2^10, 1000) % hasil: 24
```

Rangkuman

Aritmetika modular adalah dasar matematika untuk banyak algoritma kriptografi. Operasi modular memenuhi sifat-sifat kongruensi, dan invers modular dapat dihitung menggunakan algoritma Euclid diperluas yang dipelajari di bab berikutnya.

Bab 3

Algoritma Euclid dan Euclid Diperluas

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Mengimplementasikan algoritma Euclid untuk menghitung FPB
- Mengimplementasikan algoritma Euclid diperluas untuk invers modular

3.1 Algoritma Euclid dan Euclid Diperluas

3.1.1 Algoritma Euclid

Algoritma Euclid menghitung $\gcd(a, b)$ secara efisien [4, 5]. Ide: $\gcd(a, b) = \gcd(b, a \bmod b)$. Proses berulang hingga $b = 0$, maka $\gcd = a$.

3.1.2 Algoritma Euclid Diperluas

Algoritma Euclid diperluas tidak hanya menghitung $\gcd(a, b)$ tetapi juga koefisien x dan y sehingga $ax + by = \gcd(a, b)$. Koefisien x memberikan invers modular dari a modulo b jika $\gcd(a, b) = 1$ [4, 5].

Listing 3.1: Algoritma Euclid Diperluas dalam C

```
1 int extended_gcd(int a, int b, int *x, int *y) {
```

```
2   if (b == 0) { *x = 1; *y = 0; return a; }
3   int x1, y1;
4   int g = extended_gcd(b, a % b, &x1, &y1);
5   *x = y1;
6   *y = x1 - (a / b) * y1;
7   return g;
8 }
```

Implementasi lengkap dalam MATLAB, C++, dan Pascal tersedia di Bab 14.

Rangkuman

Algoritma Euclid menghitung $\gcd(a, b)$ secara efisien. Algoritma Euclid diperluas menghasilkan koefisien x, y sehingga $ax + by = \gcd(a, b)$, yang digunakan untuk menghitung invers modular—komponen penting dalam RSA.

Bab 4

Teori Bilangan: Prima dan Fast Exponentiation

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menentukan bilangan prima dan uji primalitas
- Menghitung pangkat modular (fast exponentiation)
- Menerapkan Teorema Fermat dan Euler dalam kriptografi

4.1 Teori Bilangan

4.1.1 Bilangan Prima

Bilangan prima adalah bilangan bulat positif yang hanya habis dibagi oleh 1 dan dirinya sendiri. Bilangan prima penting dalam kriptografi karena sulit memfaktorkan bilangan besar menjadi faktor prima [8, 4, 5].

4.1.2 Teorema Fermat Kecil

Jika p prima dan $\gcd(a, p) = 1$, maka $a^{p-1} \equiv 1 \pmod{p}$. Konsekuensi: $a^{-1} \equiv a^{p-2} \pmod{p}$.

4.1.3 Fungsi Euler

Fungsi Euler $\phi(n)$ menyatakan banyaknya bilangan bulat positif kurang dari n yang relatif prima dengan n . Untuk $n = pq$ dengan p, q prima: $\phi(n) = (p-1)(q-1)$. Teorema Euler: jika $\gcd(a, n) = 1$, maka $a^{\phi(n)} \equiv 1 \pmod{n}$ [8, 4, 5].

4.2 Pangkat Modular (Fast Exponentiation)

Menghitung $a^b \bmod n$ secara naif dengan mengalikan a sebanyak b kali membutuhkan waktu $O(b)$. **Square-and-multiply** (atau binary exponentiation) mengurangi kompleksitas menjadi $O(\log b)$ [5, 4].

Tabel 4.1: Contoh: $3^{11} \bmod 5 = 2$ ($11 = 1011_2$)

Langkah	Operasi
Biner b	$11 = 1011_2$
Square-and-multiply	$3^1 \rightarrow 3^2 \rightarrow 3^5 \rightarrow 3^{11}$
Hasil	$3^{11} \equiv 2 \pmod{5}$

Algoritma: representasikan b dalam biner. Untuk setiap bit, kuadratkan basis; jika bit 1, kalikan hasil dengan basis.

Listing 4.1: Fast Modular Exponentiation dalam C

```

1 long long mod_pow(long long base, long long exp, long long mod) {
2     long long result = 1;
3     base = base % mod;
4     while (exp > 0) {
5         if (exp % 2 == 1)
6             result = (result * base) % mod;
7         exp = exp / 2;

```

```
8         base = (base * base) % mod;
9     }
10     return result;
11 }
```

Implementasi lengkap tersedia di Bab 14 dalam C, MATLAB, C++, dan Pascal.

Rangkuman

Teori bilangan—bilangan prima, Teorema Fermat, Fungsi Euler—merupakan fondasi matematika RSA. Fast modular exponentiation (square-and-multiply) memungkinkan komputasi $a^b \bmod n$ secara efisien, yang esensial untuk enkripsi dan dekripsi RSA.

Bab 5

Cipher Caesar dan Substitusi Sederhana

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Mengimplementasikan cipher Caesar dan substitusi monoalfabetik
- Menganalisis kelemahan cipher klasik
- Melakukan enkripsi dan dekripsi dengan cipher substitusi

5.1 Cipher Caesar

Cipher Caesar adalah cipher substitusi monoalfabetik paling sederhana [6, 8]. Setiap huruf digeser sejumlah posisi tetap dalam alfabet. Dengan shift k , huruf A menjadi huruf ke- $(1 + k) \bmod 26$.

Contoh dengan shift 3: HELLO $\rightarrow$ KHOOR. Dekripsi dilakukan dengan shift kebalikan $(26 - k)$.

Tabel 5.1: Tabel Caesar cipher dengan shift 3 (A→D, B→E, ...)

Plain	A	B	C	D	E	F	G	H	I	J	K	L	M
Cipher	D	E	F	G	H	I	J	K	L	M	N	O	P
Plain	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Listing 5.1: Caesar Cipher dalam C (ringkas)

```

1 char caesar(char c, int shift) {
2     if (c >= 'A' && c <= 'Z')
3         return 'A' + (c - 'A' + shift) % 26;
4     if (c >= 'a' && c <= 'z')
5         return 'a' + (c - 'a' + shift) % 26;
6     return c;
7 }

```

Listing 5.2: Caesar Cipher dalam Pascal (ringkas)

```

1 function caesar_encrypt(s: string; k: integer): string;
2 var i: integer; result: string;
3 begin
4     result := '';
5     for i := 1 to length(s) do
6         if (s[i] >= 'A') and (s[i] <= 'Z') then
7             result := result + chr(ord('A') + (ord(s[i]) - ord('A') + k)
8                 mod 26)
9         else
10             result := result + s[i];
11     caesar_encrypt := result;
12 end;

```

Kelemahan: hanya 26 kemungkinan kunci; mudah diserang dengan brute force. Implementasi lengkap dalam MATLAB dan C++ tersedia di Bab 14.

5.2 Substitusi Monoalfabetik

Cipher substitusi monoalfabetik mengganti setiap huruf dengan huruf lain secara tetap [8, 6]. Pemetaan adalah permutasi dari 26 huruf alfabet. Ada $26!$ kemungkinan kunci—secara teoritis sangat besar, tetapi rentan terhadap **frequency analysis** karena frekuensi huruf dalam bahasa (misalnya E, A, T paling sering dalam bahasa Inggris) dapat dieksploitasi.

Rangkuman

Cipher Caesar dan substitusi monoalfabetik merupakan cipher klasik paling sederhana. Caesar hanya memiliki 26 kunci; substitusi umum memiliki $26!$ kunci tetapi rentan terhadap frequency analysis. Implementasi dalam empat bahasa pemrograman tersedia di Bab 14.

Bab 6

Cipher Vigenère dan Polyalphabetic

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Mengimplementasikan cipher Vigenère
- Menganalisis kelemahan cipher polyalphabetic
- Memahami cipher transposisi sebagai pelengkap

6.1 Cipher Vigenère

Cipher Vigenère adalah cipher polyalphabetic yang menggunakan kata kunci berulang. Setiap huruf plaintext digeser sesuai huruf kunci yang bersesuaian. Dengan kunci “KEY”, huruf pertama digeser K (10), huruf kedua E (4), huruf ketiga Y (24), lalu berulang.

Tabel 6.1: Contoh Vigenère: plaintext “HELLO”, kunci “KEY”

Plaintext	H	E	L	L	O
Kunci	K	E	Y	K	E
Ciphertext	R	I	J	V	S

Rumus: $C_i = (P_i + K_i) \bmod 26$, dengan P_i , K_i adalah posisi huruf (A=0).

Listing 6.1: Vigenère Cipher dalam MATLAB

```

1 function c = vigenere_encrypt(plain, key)
2     plain = upper(plain); key = upper(key);
3     c = '';
4     for i = 1:length(plain)
5         if plain(i) >= 'A' && plain(i) <= 'Z'
6             ki = mod(i-1, length(key)) + 1;
7             p = double(plain(i)) - 65;
8             k = double(key(ki)) - 65;
9             c = [c char(mod(p+k, 26) + 65)];
10        else, c = [c plain(i)]; end
11    end
12 end

```

Lebih kuat dari Caesar karena satu huruf plaintext dapat dipetakan ke banyak huruf ciphertext. Namun rentan terhadap **Kasiski examination** jika panjang kunci dapat diperkirakan [6, 8].

6.2 Transposisi

Cipher transposisi mengubah urutan huruf tanpa mengubah huruf itu sendiri. Contoh: rail fence—tuliskan plaintext dalam zigzag pada beberapa baris, lalu baca per baris. Contoh lain: columnar transposition—tuliskan plaintext dalam matriks, lalu baca per kolom menurut urutan kunci.

Transposisi saja lemah terhadap analisis frekuensi karena huruf tidak berubah. Sering dikombinasikan dengan substitusi untuk keamanan lebih baik [2, 8].

Rangkuman

Cipher Vigenère menggunakan kata kunci berulang sehingga satu huruf plaintext dapat dipetakan ke banyak huruf ciphertext—lebih kuat dari Caesar. Namun rentan terhadap Kasiski examination. Cipher transposisi mengubah urutan huruf; sering dikombinasikan dengan substitusi.

Bab 7

Block Cipher: DES, AES, dan Mode Operasi

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menjelaskan struktur DES dan AES
- Membedakan mode operasi block cipher (ECB, CBC)
- Memahami prinsip block cipher modern

7.1 Block Cipher, DES, dan AES

7.1.1 Block Cipher

Block cipher mengenkripsi data dalam blok-blok tetap (misalnya 64 atau 128 bit). Algoritma seperti DES dan AES adalah block cipher [2, 6]. Operasi dasar meliputi substitusi (S-box), permutasi (P-box), dan operasi XOR dengan subkunci.

Tabel 7.1: Perbandingan DES dan AES

Aspek	DES	AES
Ukuran blok	64 bit	128 bit
Ukuran kunci	56 bit	128, 192, 256 bit
Jumlah putaran	16 (Feistel)	10, 12, 14
Struktur	Feistel network	SPN (Substitution-Permutation)
Status	Tidak disarankan	Standar saat ini

7.1.2 DES

Data Encryption Standard (DES) dikembangkan IBM dan diadopsi NIST pada 1977 [9]. Menggunakan blok 64 bit dan kunci 56 bit. Struktur: 16 putaran Feistel, setiap putaran menggunakan S-box dan permutasi.

DES dianggap tidak aman untuk aplikasi modern karena ukuran kunci 56 bit terlalu kecil terhadap serangan brute force. Triple DES (3DES) menggunakan DES tiga kali dengan kunci berbeda untuk meningkatkan keamanan [2, 6].

7.1.3 AES

Advanced Encryption Standard (AES) menggantikan DES pada 2001 [10]. Menggunakan blok 128 bit dan kunci 128, 192, atau 256 bit. AES-128 adalah pilihan paling umum.

Struktur: matriks state 4×4 byte. Setiap putaran meliputi SubBytes (S-box), ShiftRows, MixColumns, dan AddRoundKey. AES-128 memiliki 10 putaran. AES dirancang untuk resistan terhadap serangan diferensial dan linear [10, 2, 6].

7.2 Mode Operasi Block Cipher

Block cipher membutuhkan **mode operasi** untuk mengenkripsi data yang lebih panjang dari ukuran blok [11, 7]. Mode yang umum: ECB, CBC, CTR, GCM.

7.2.1 ECB (Electronic Codebook)

Setiap blok plaintext dienkripsi secara independen. Kelemahan: blok identik menghasilkan ciphertext identik—rentan terhadap analisis pola [11].

7.2.2 CBC (Cipher Block Chaining)

Setiap blok plaintext di-XOR dengan ciphertext blok sebelumnya sebelum enkripsi. Memerlukan IV (Initialization Vector) untuk blok pertama. Lebih aman dari ECB karena ciphertext bergantung pada semua blok sebelumnya [11, 7].

Rangkuman

Block cipher mengenkripsi data dalam blok tetap. DES (56-bit kunci) telah digantikan AES (128/192/256-bit). Mode operasi ECB dan CBC menentukan cara mengenkripsi data yang lebih panjang dari ukuran blok; CBC lebih aman dari ECB.

Bab 8

Algoritma RSA

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menjelaskan konsep kriptografi kunci publik
- Mengimplementasikan algoritma RSA
- Menghitung enkripsi dan dekripsi RSA menggunakan fast exponentiation

8.1 Konsep Kunci Publik

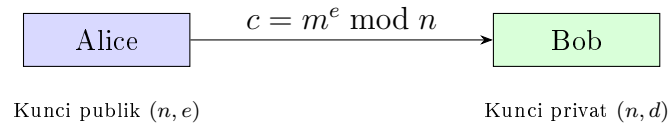
Kriptografi kunci publik menggunakan sepasang kunci: **kunci publik** (dapat dibagikan) dan **kunci privat** (dijaga rahasia). Apa yang dienkripsi dengan kunci publik hanya dapat didekripsi dengan kunci privat, dan sebaliknya [1, 12].

Keuntungan utama: tidak perlu saluran aman untuk pertukaran kunci. Siapa pun dapat mengenkripsi pesan untuk Bob menggunakan kunci publik Bob; hanya Bob yang dapat mendekripsi dengan kunci privatnya.

Kelemahan: lebih lambat dibanding kriptografi simetris. Oleh karena itu, sistem hibrida sering digunakan: kunci simetris dipertukarkan dengan kriptografi asimetris, lalu data dienkripsi dengan kunci simetris.

8.2 Algoritma RSA

RSA (Rivest–Shamir–Adleman, 1977) adalah algoritma kriptografi asimetris paling luas digunakan [13, 1, 5]. Berdasarkan kesulitan memfaktorkan bilangan besar.



Gambar 8.1: Alur enkripsi RSA: Alice mengenkripsi dengan kunci publik Bob

8.2.1 Generasi Kunci

1. Pilih dua bilangan prima besar p dan q .
2. Hitung $n = pq$ dan $\phi(n) = (p - 1)(q - 1)$.
3. Pilih e dengan $1 < e < \phi(n)$ dan $\gcd(e, \phi(n)) = 1$.
4. Hitung d sehingga $ed \equiv 1 \pmod{\phi(n)}$ (gunakan Euclid diperluas).
5. Kunci publik: (n, e) . Kunci privat: (n, d) .

8.2.2 Enkripsi dan Dekripsi

Enkripsi: $c = m^e \bmod n$. Dekripsi: $m = c^d \bmod n = m^{ed} \bmod n = m \pmod{n}$ (berdasarkan Teorema Euler) [13, 1, 5].

Listing 8.1: Enkripsi RSA dalam C (simplified)

```

1 // Contoh: enkripsi m=3 dengan n=33, e=3
2 // c = 3^3 mod 33 = 27
3 long long rsa_encrypt(long long m, long long e, long long n) {
4     return mod_pow(m, e, n); // gunakan fast exponentiation
5 }
6 long long rsa_decrypt(long long c, long long d, long long n) {
7     return mod_pow(c, d, n);
8 }
  
```


Implementasi lengkap RSA menggunakan fast modular exponentiation tersedia di Bab 14.

Rangkuman

RSA adalah algoritma kriptografi asimetris paling luas digunakan, berdasarkan kesulitan memfaktorkan bilangan besar. Generasi kunci melibatkan pemilihan prima p, q , perhitungan $\phi(n)$, dan penentuan e, d . Enkripsi dan dekripsi menggunakan fast modular exponentiation.

Bab 9

Fungsi Hash (SHA, MD5)

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menjelaskan sifat fungsi hash kriptografis
- Mengimplementasikan SHA/MD5 untuk integritas data
- Membedakan fungsi hash yang aman dan tidak aman

9.1 Fungsi Hash

Fungsi hash kriptografis memetakan input berukuran sembarang ke output berukuran tetap (misalnya 256 bit) [1, 5]. Sifat penting:

Tabel 9.1: Sifat fungsi hash kriptografis

Sifat	Deskripsi
Preimage resistance	Sulit menemukan x jika diberikan $h(x)$
Second preimage	Sulit menemukan $x' \neq x$ dengan $h(x') = h(x)$
Collision resistance	Sulit menemukan $x \neq x'$ dengan $h(x) = h(x')$

- **Preimage resistance:** Sulit menemukan x jika diberikan $h(x)$.

- **Second preimage resistance:** Sulit menemukan $x' \neq x$ dengan $h(x') = h(x)$.
- **Collision resistance:** Sulit menemukan x, x' dengan $x \neq x'$ dan $h(x) = h(x')$.

Aplikasi: verifikasi integritas file, penyimpanan kata sandi, tanda tangan digital [1, 5].

9.2 SHA dan MD5

9.2.1 MD5

Message Digest 5 menghasilkan hash 128 bit. Saat ini dianggap tidak aman karena rentan terhadap collision attack. Tidak disarankan untuk aplikasi keamanan kritis [2].

9.2.2 SHA (Secure Hash Algorithm)

Keluarga SHA dikembangkan NIST [14]. SHA-256 menghasilkan 256 bit, SHA-384 dan SHA-512 menghasilkan hash lebih panjang. SHA-256 adalah standar untuk integritas data dan tanda tangan digital. SHA-3 (Keccak) adalah varian terbaru dengan desain berbeda [14, 2].

Rangkuman

Fungsi hash memetakan input sembarang ke output berukuran tetap. Sifat penting: preimage resistance, second preimage resistance, collision resistance. SHA-256 adalah standar untuk integritas data; MD5 tidak disarankan untuk keamanan kritis.

Bab 10

Tanda Tangan Digital

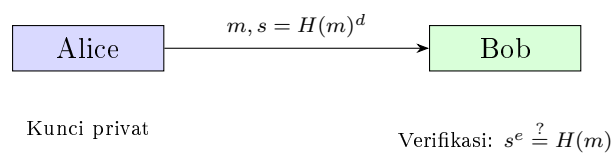
Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Merancang tanda tangan digital berbasis RSA
- Memverifikasi keaslian dan integritas dokumen digital
- Memahami non-repudiation dalam tanda tangan digital

10.1 Tanda Tangan Digital

Tanda tangan digital memastikan keaslian dan integritas dokumen digital [13, 1, 7]. Prinsip: pengirim mengenkripsi hash dokumen dengan kunci privat; penerima mendekripsi dengan kunci publik pengirim dan memverifikasi hash cocok dengan hash dokumen yang dihitung ulang.



Gambar 10.1: Alur tanda tangan digital berbasis RSA

Skema berbasis RSA: $\text{Sign}(m) = H(m)^d \bmod n$; $\text{Verify}(m, s) = (s^e \bmod n \stackrel{?}{=} H(m))$.

Tanda tangan digital memberikan non-repudiation: pengirim tidak dapat menyangkal telah menandatangani [13, 1, 7].

Rangkuman

Tanda tangan digital memastikan keaslian dan integritas dokumen digital dengan mengenkripsi hash dokumen menggunakan kunci privat pengirim. Skema berbasis RSA memberikan non-repudiation: pengirim tidak dapat menyangkal telah menandatangani.

Bab 11

Protokol Kriptografi

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menganalisis protokol Diffie-Hellman
- Memahami protokol pertukaran kunci yang aman
- Menjelaskan prinsip protokol keamanan lapisan transport (TLS/SSL)

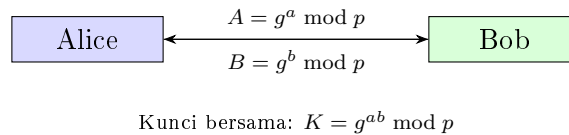
11.1 Protokol Kriptografi

Protokol kriptografi mendefinisikan urutan langkah dan format pesan untuk mencapai tujuan keamanan [7, 2]. Contoh: protokol pertukaran kunci (Diffie-Hellman), protokol autentikasi (Kerberos), protokol keamanan lapisan transport (TLS/SSL).

Protokol yang baik harus memenuhi: kerahasiaan, integritas, autentikasi, dan non-repudiation sesuai kebutuhan aplikasi.

11.2 Protokol Diffie-Hellman

Protokol Diffie-Hellman (1976) memungkinkan dua pihak menetapkan kunci rahasia bersama melalui saluran tidak aman [12, 1, 8]. Berdasarkan masalah diskret logaritma.



Gambar 11.1: Protokol Diffie-Hellman: pertukaran nilai untuk menghasilkan kunci rahasia

Alice dan Bob menyepakati bilangan prima p dan generator g . Alice memilih a , mengirim $A = g^a \bmod p$. Bob memilih b , mengirim $B = g^b \bmod p$. Keduanya menghitung $K = g^{ab} \bmod p$ (Alice: B^a , Bob: A^b). Eve yang menyadap A dan B tidak dapat menghitung K tanpa menyelesaikan diskret logaritma [12, 1, 8].

Listing 11.1: Diffie-Hellman dalam MATLAB

```

1 function K = diffie_hellman(p, g, a, b)
2     A = mod_pow(g, a, p); % Alice mengirim A
3     B = mod_pow(g, b, p); % Bob mengirim B
4     K_alice = mod_pow(B, a, p); % K = B^a mod p
5     K_bob = mod_pow(A, b, p); % K = A^b mod p
6     K = K_alice; % K_alice == K_bob
7 end

```

Rangkuman

Protokol kriptografi mendefinisikan urutan langkah untuk mencapai tujuan keamanan. Protokol Diffie-Hellman memungkinkan pertukaran kunci rahasia melalui saluran tidak aman berdasarkan masalah diskret logaritma. Protokol yang baik memenuhi kerahasiaan, integritas, autentikasi, dan non-repudiation.

Bab 12

Manajemen Kunci

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Menjelaskan siklus hidup manajemen kunci
- Merancang protokol pertukaran kunci yang aman
- Memahami Infrastruktur Kunci Publik (PKI) dan sertifikat digital

12.1 Manajemen Kunci

Manajemen kunci mencakup siklus hidup kunci: **generasi** (menggunakan RNG kriptografis), **distribusi** (pertukaran aman), **penyimpanan** (hardware security module, enkripsi), **rotasi** (penggantian berkala), dan **pemusnahan** (penghapusan aman) [2, 7, 5].

Infrastruktur Kunci Publik (PKI) menyediakan sertifikat digital untuk mengikat identitas dengan kunci publik. Certificate Authority (CA) menandatangani sertifikat yang memverifikasi kepemilikan kunci [2, 7, 5].

Rangkuman

Manajemen kunci mencakup generasi, distribusi, penyimpanan, rotasi, dan pemusnahan kunci. PKI menyediakan sertifikat digital untuk mengikat identitas dengan kunci publik melalui Certificate Authority (CA).

Bab 13

Kriptanalisis

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Melakukan frequency analysis pada cipher substitusi
- Menyerang cipher Vigenère dengan Kasiski examination
- Mengevaluasi kekuatan algoritma terhadap berbagai jenis serangan

13.1 Frequency Analysis

Frequency analysis mengeksploitasi fakta bahwa huruf dalam bahasa alami memiliki frekuensi kemunculan yang dapat diprediksi [8, 6].

Tabel 13.1: Frekuensi huruf dalam bahasa Inggris (%)

E 12.7	T 9.1	A 8.2	O 7.5	I 7.0	N 6.7	S 6.3
H 6.1	R 6.0	D 4.3	L 4.0	C 2.8	U 2.8	M 2.4
W 2.4	F 2.2	G 2.0	Y 2.0	P 1.9	B 1.5	V 1.0
K 0.8	J 0.2	X 0.2	Q 0.1	Z 0.1		

Dalam bahasa Inggris, E, T, A, O, I, N paling sering muncul; J, Q, X, Z paling jarang.

Untuk cipher substitusi monoalfabetik: hitung frekuensi setiap huruf dalam ciphertext, bandingkan dengan frekuensi standar bahasa, dan cocokkan pemetaan [8, 6]. Ciphertext yang lebih panjang memberikan estimasi frekuensi lebih akurat.

13.2 Kasiski Examination

Kasiski examination menyerang cipher Vigenère dengan menemukan panjang kunci [8, 6]. Ide: jika substring plaintext berulang pada jarak kelipatan panjang kunci, substring ciphertext yang sama akan muncul. Cari pengulangan substring dalam ciphertext; jarak antar pengulangan kemungkinan kelipatan panjang kunci. GCD dari jarak-jarak tersebut memberikan estimasi panjang kunci. Setelah panjang kunci diketahui, setiap posisi dapat dianalisis sebagai cipher Caesar terpisah.

13.3 Serangan Kriptanalisis

Jenis serangan kriptanalisis:

- **Brute force:** Mencoba semua kemungkinan kunci. Efektif jika ruang kunci kecil (misalnya Caesar).
- **Known-plaintext:** Memanfaatkan pasangan plaintext–ciphertext untuk memperoleh kunci.
- **Chosen-plaintext:** Penyerang memilih plaintext dan memperoleh ciphertext.
- **Cryptanalysis:** Serangan matematis spesifik (misalnya diferensial, linear pada block cipher) [5, 6].

Evaluasi kekuatan algoritma mempertimbangkan ukuran ruang kunci, resistansi terhadap serangan matematis, dan keamanan implementasi [5, 6].

Rangkuman

Kriptanalisis mempelajari cara menyerang sistem kriptografi. Frequency analysis mengeksploitasi pola frekuensi huruf; Kasiski examination menemukan panjang kunci Vigenère.

Evaluasi kekuatan algoritma mempertimbangkan ruang kunci dan resistansi terhadap serangan matematis.

Bab 14

Implementasi Multi-Bahasa

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

- Mengimplementasikan cipher dasar dalam MATLAB, C, C++, dan Pascal
- Membuat dokumentasi dan pengujian kode
- Membandingkan implementasi antar bahasa pemrograman

14.1 Implementasi dalam MATLAB

Berikut implementasi Caesar Cipher dan Algoritma Euclid Diperluas dalam MATLAB [7, 6].

14.1.1 Caesar Cipher

Listing 14.1: Implementasi Caesar Cipher dalam MATLAB

```
1 function ciphertext = caesar_encrypt(plaintext, shift)
2 % Caesar Cipher Encryption
3 % plaintext: string input
4 % shift: integer (0-25)
5     plaintext = upper(plaintext);
```

```

6      ciphertext = '';
7      for i = 1:length(plaintext)
8          if plaintext(i) >= 'A' && plaintext(i) <= 'Z'
9              c = double(plaintext(i)) - 65;
10             c = mod(c + shift, 26);
11             ciphertext = [ciphertext char(c + 65)];
12         else
13             ciphertext = [ciphertext plaintext(i)];
14         end
15     end
16 end

17
18 function plaintext = caesar_decrypt(ciphertext, shift)
19     plaintext = caesar_encrypt(ciphertext, 26 - shift);
20 end

```

14.1.2 Algoritma Euclid Diperluas

Listing 14.2: Extended Euclidean Algorithm dalam MATLAB

```

1 function [g, x, y] = extended_gcd(a, b)
2 % Extended Euclidean Algorithm: ax + by = gcd(a,b)
3     if b == 0
4         g = a;
5         x = 1;
6         y = 0;
7     else
8         [g, x1, y1] = extended_gcd(b, mod(a, b));
9         x = y1;
10        y = x1 - floor(a/b) * y1;
11    end
12 end
13
14 % Contoh: invers modular dari a mod m
15 function inv = mod_inverse(a, m)

```



```

16     [g, x, ~] = extended_gcd(mod(a, m), m);
17     if g ~= 1
18         inv = []; % tidak ada invers
19     else
20         inv = mod(x, m);
21     end
22 end

```

14.2 Implementasi dalam C

Berikut implementasi Caesar Cipher dan Fast Modular Exponentiation dalam C.

14.2.1 Caesar Cipher

Listing 14.3: Implementasi Caesar Cipher dalam C

```

1 #include <stdio.h>
2 #include <ctype.h>
3 #include <string.h>
4
5 void caesar_encrypt(char *plaintext, int shift) {
6     int i;
7     for (i = 0; plaintext[i] != '\0'; i++) {
8         if (isalpha(plaintext[i])) {
9             char base = isupper(plaintext[i]) ? 'A' : 'a';
10            plaintext[i] = base + (plaintext[i] - base + shift) %
11                           26;
12        }
13    }
14
15 void caesar_decrypt(char *ciphertext, int shift) {
16     caesar_encrypt(ciphertext, 26 - shift);
17 }
18

```

```
19 int main() {
20     char msg[] = "HELLO CRYPTO";
21     caesar_encrypt(msg, 3);
22     printf("Encrypted: %s\n", msg);
23     caesar_decrypt(msg, 3);
24     printf("Decrypted: %s\n", msg);
25     return 0;
26 }
```

14.2.2 Fast Modular Exponentiation

Listing 14.4: Fast Modular Exponentiation dalam C

```
1  #include <stdio.h>
2
3  long long mod_pow(long long base, long long exp, long long mod) {
4      long long result = 1;
5      base = base % mod;
6      while (exp > 0) {
7          if (exp % 2 == 1)
8              result = (result * base) % mod;
9          exp = exp / 2;
10         base = (base * base) % mod;
11     }
12     return result;
13 }
14
15 int main() {
16     printf("2^10 mod 1000 = %lld\n", mod_pow(2, 10, 1000));
17     printf("3^644 mod 645 (RSA) = %lld\n", mod_pow(3, 644, 645));
18     return 0;
19 }
```

14.3 Implementasi dalam C++

Berikut implementasi Caesar Cipher dalam C++.

Listing 14.5: Implementasi Caesar Cipher dalam C++

```
1 #include <iostream>
2 #include <string>
3 #include <cctype>
4
5 std::string caesar_encrypt(std::string plaintext, int shift) {
6     std::string ciphertext = "";
7     for (char c : plaintext) {
8         if (isalpha(c)) {
9             char base = isupper(c) ? 'A' : 'a';
10            ciphertext += base + (c - base + shift + 26) % 26;
11        } else {
12            ciphertext += c;
13        }
14    }
15    return ciphertext;
16 }
17
18 std::string caesar_decrypt(std::string ciphertext, int shift) {
19     return caesar_encrypt(ciphertext, 26 - shift);
20 }
21
22 int main() {
23     std::string msg = "HELLO CRYPTO";
24     std::cout << "Encrypted: " << caesar_encrypt(msg, 3) << std::
25         endl;
26     std::cout << "Decrypted: " << caesar_decrypt(
27         caesar_encrypt(msg, 3), 3) << std::endl;
28     return 0;
29 }
```

14.4 Implementasi dalam Pascal

Berikut implementasi Caesar Cipher dalam Pascal.

Listing 14.6: Implementasi Caesar Cipher dalam Pascal

```
1 program CaesarCipher;
2
3 var
4     plaintext, ciphertext: string;
5     shift, i: integer;
6     c: char;
7
8 function caesar_encrypt(s: string; k: integer): string;
9 var
10    result: string;
11    i: integer;
12    base: char;
13 begin
14    result := '';
15    for i := 1 to length(s) do
16    begin
17        if (s[i] >= 'A') and (s[i] <= 'Z') then
18        begin
19            base := 'A';
20            result := result + chr(ord(base) + (ord(s[i]) - ord(base) + k
21                ) mod 26);
22        end
23        else if (s[i] >= 'a') and (s[i] <= 'z') then
24        begin
25            base := 'a';
26            result := result + chr(ord(base) + (ord(s[i]) - ord(base) + k
27                ) mod 26);
28        end
29        else
30            result := result + s[i];
```

```
29     end;
30     caesar_encrypt := result;
31 end;
32
33 function caesar_decrypt(s: string; k: integer): string;
34 begin
35     caesar_decrypt := caesar_encrypt(s, 26 - k);
36 end;
37
38 begin
39     plaintext := 'HELLO CRYPTO';
40     shift := 3;
41     ciphertext := caesar_encrypt(plaintext, shift);
42     writeln('Encrypted: ', ciphertext);
43     writeln('Decrypted: ', caesar_decrypt(ciphertext, shift));
44 end.
```

Rangkuman

Bab ini menyajikan implementasi Caesar Cipher, Algoritma Euclid Diperluas, dan Fast Modular Exponentiation dalam empat bahasa pemrograman. Mahasiswa diharapkan dapat mengadaptasi kode untuk proyek implementasi algoritma kriptografi lainnya.

Daftar Referensi

Bibliografi

- [1] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3 edition, 2020.
- [2] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 7 edition, 2017.
- [3] Claude E. Shannon. Communication theory of secrecy systems. *Bell System Technical Journal*, 28(4):656–715, 1949.
- [4] Douglas R. Stinson and Maura Paterson. *Cryptography: Theory and Practice*. CRC Press, 4 edition, 2018.
- [5] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.
- [6] Bruce Schneier. *Applied Cryptography*. John Wiley & Sons, 2 edition, 1996.
- [7] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. *Cryptography Engineering*. Wiley, 2010.
- [8] Wade Trappe and Lawrence C. Washington. *Introduction to Cryptography with Coding Theory*. Pearson, 2 edition, 2006.
- [9] National Institute of Standards and Technology. FIPS 46-3: Data encryption standard (DES). <https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf>, 1999. Historical. Diakses 2024.
- [10] National Institute of Standards and Technology. FIPS 197: Advanced encryption standard (AES). <https://csrc.nist.gov/pubs/fips/197/final>, 2001. Diakses 2024.

- [11] National Institute of Standards and Technology. SP 800-38a: Recommendation for block cipher modes of operation. <https://csrc.nist.gov/pubs/sp/800/38/a/final>, 2001. ECB, CBC, CFB, OFB, CTR. Diakses 2024.
- [12] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [13] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [14] National Institute of Standards and Technology. FIPS 180-4: Secure hash standard (SHS). <https://csrc.nist.gov/pubs/fips/180-4/final>, 2015. DOI: 10.6028/NIST.FIPS.180-4. Diakses 2024.